

Atividade A3 - Grafos
Maria Júlia de Melo Honorato - 23200368

Os dois grafos foram feitos adaptando os grafos usados nos relatórios anteriores.

Exercício 1 - Edmonds Karp: O algoritmo utilizado foi baseado nas anotações da disciplina.

- Vetores C, A: Vetores conhecido e antecessor, respectivamente.
 - Uso de $n+1$ para permitir indexação direta, posição 0 não é utilizada (definição mantida no outro Exercício)
- `std::queue<int>` Q: Fila de inteiros utilizada para organizar a ordem dos vértices.
- `std::vector<int>` p: Vetor de inteiros para reconstruir e guardar a sequência de vértices do caminho aumentante.
- `std::vector<std::vector<double>>` cf: Vetor de vetores que representa as capacidades residuais. Cada índice representa um vértice de origem e guarda um vetor com as capacidades residuais para cada vértice de destino.
- `std::vector<int>` vizinhos: Vetor para armazenar os vizinhos do vértice (vai ser usado no outro Exercício).
- `std::numeric_limits<double>::infinity`: Função de limits que retorna o valor infinito positivo, usado para marcar se é inacessível.

Exercício 2 - Hopcroft Karp: O algoritmo utilizado foi baseado nas anotações da disciplina.

- `std::queue<int>` Q: Fila de inteiros para organizar a ordem dos vértices.
- `std::numeric_limits<int>::max()`: Função de limits que retorna o maior valor inteiro positivo possível, usado para marcar se é inacessível. Foi utilizado esse no lugar de `infinity` porque, do jeito que estava fazendo, era mais fácil trabalhar com `max` que já estava usando inteiros ao invés de `double`.
- `std::vector<int>` vizinhos: Mesma coisa do Edmonds Karp.
- `std::vector<int>` X: Vetor de inteiros para armazenar vértices da partição X.
- `std::vector<int>` mate: Vetor de inteiros que armazena o emparelhamento dos vértices.
- `std::vector<int>` D: Vetor de inteiros que armazena as distâncias de cada vértice a partir dos livres em X.
- `std::fill`: Usado para resetar o vetor de distâncias antes da próxima iteração.